

Understanding Distributed Cloud Computing, Edge Computing, Containers, Docker, Kubernetes, and DevOps

Aditya Thombre

Roll No. 04

CSE 6th SEM

What is Distributed Cloud Computing?

- - Definition: A model where cloud services are distributed across different physical locations while remaining centrally managed.
- - Key Benefits:
 - • Enhanced scalability
 - • Improved performance and latency
 - • Regulatory compliance
- - Use Cases: Multi-cloud strategies, disaster recovery, global applications.

What is Edge Computing?

- - Definition: A computing paradigm that brings computation and data storage closer to the location where it is needed.
- - Key Benefits:
 - • Reduced latency
 - • Lower bandwidth costs
 - • Real-time processing
- - Use Cases: IoT, autonomous vehicles, industrial automation.

Distributed Cloud vs Edge Computing

- - Distributed Cloud: Centralized management, scalable, used for multi-cloud services.
- - Edge Computing: Decentralized, ultra-low latency, used for real-time processing.

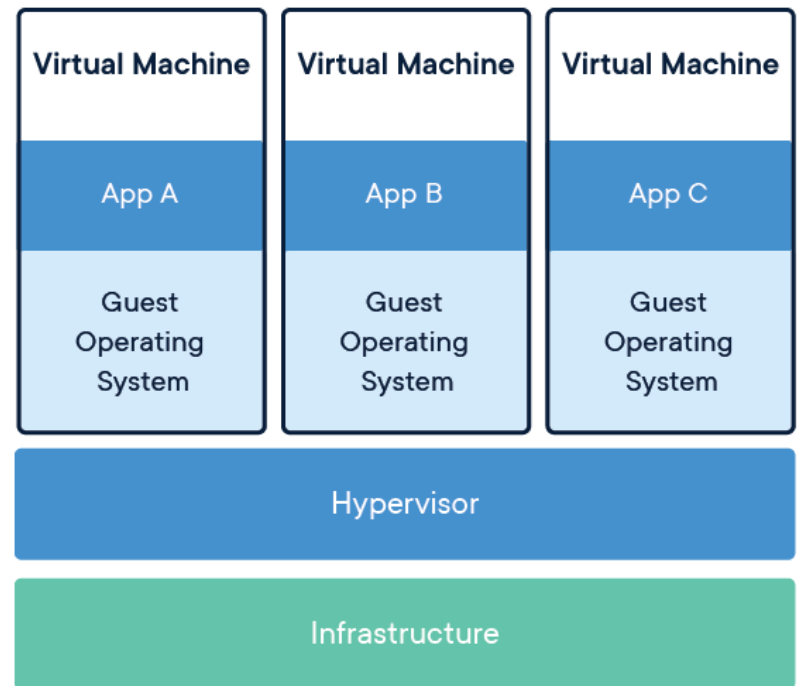
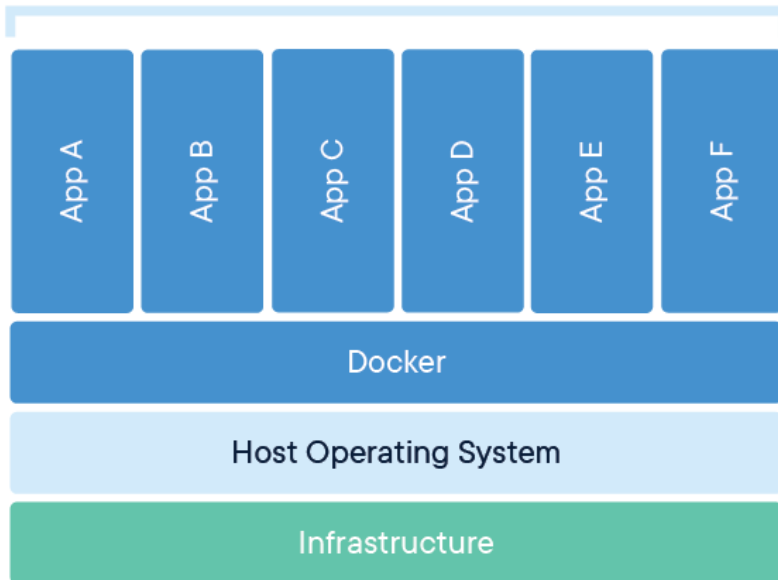
What are Containers?

- - Lightweight, portable, and isolated units that run applications and their dependencies.
- - Key Benefits:
 - • Consistency across environments
 - • Faster deployment
 - • Scalability.

CONTAINERS



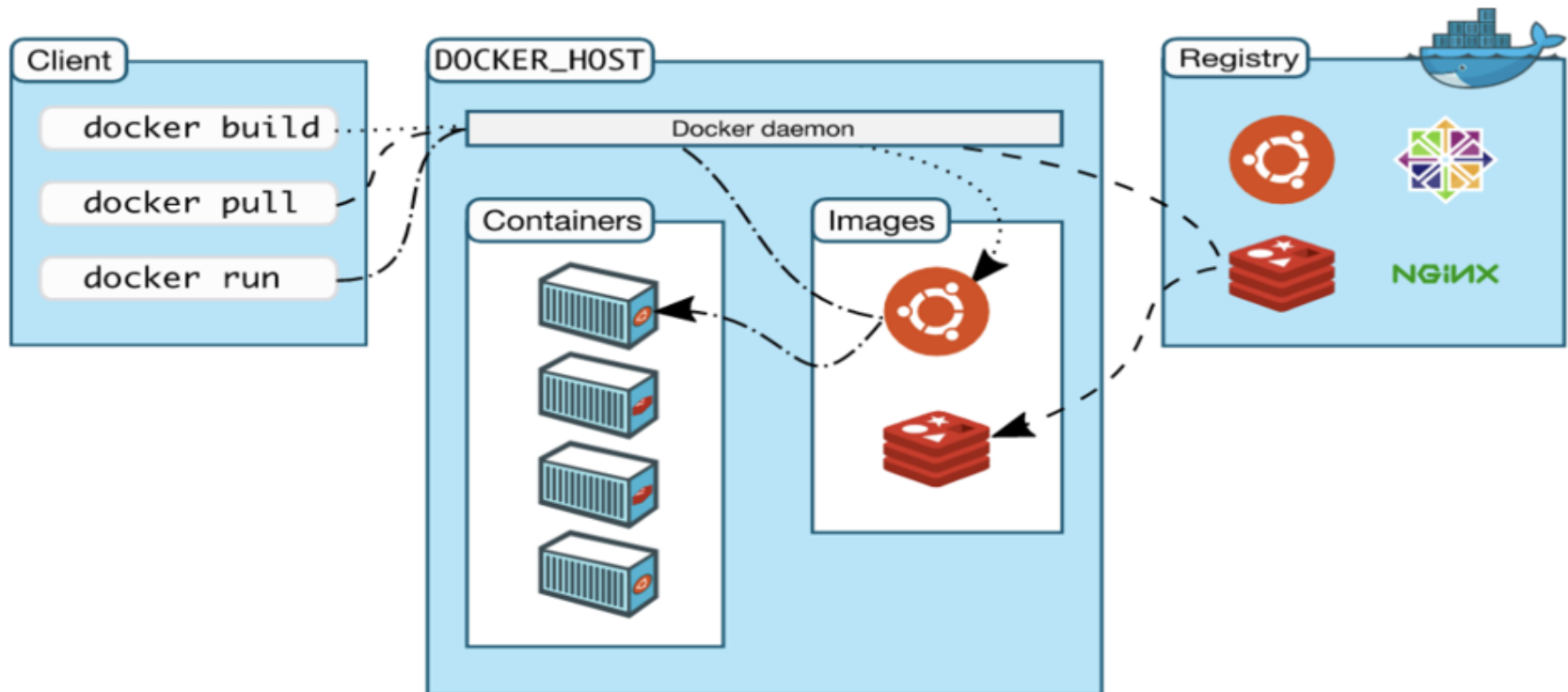
Containerized Applications



Docker Overview

- - A platform for creating, deploying, and running applications in containers.
- - Why Docker?
 - • Easy to set up and use
 - • Environment consistency
 - • Efficient resource utilization.

WORKING OF DOCKER



Kubernetes Overview

- - Open-source container orchestration platform for managing, scaling, and deploying containers.
- - Key Features:
 - • Automated scaling
 - • Self-healing capabilities
 - • Load balancing.

Docker vs Kubernetes

- - Docker: Containerization, limited scalability, manual deployment.
- - Kubernetes: Orchestration, highly scalable, automated deployment.



What is DevOps?

- - A combination of practices and tools integrating software development and IT operations.
- - Core Principles:
 - • Collaboration
 - • Automation
 - • Continuous integration and deployment (CI/CD).

DevOps Lifecycle

- 1. Plan: Define requirements and project scope.
- 2. Develop: Write and test code collaboratively.
- 3. Build: Package applications using CI/CD pipelines.
- 4. Test: Automated and manual testing.
- 5. Release & Deploy: Controlled application deployment.
- 6. Operate & Monitor: Continuous monitoring and improvement.

Summary

- - Distributed Cloud vs Edge Computing: Centralized vs decentralized.
- - Containers & Docker: Lightweight application deployment.
- - Kubernetes: Container orchestration.
- - DevOps: Streamlined software development and deployment.



THANK YOU

RN. 04

